CS-2001
Data Structures
Fall 2023

Graph

Rizwan Ul Haq

Assistant Professor

FAST-NU

rizwan.haq@nu.edu.pk

Graph definitions

- A graph is a collection of nodes (or vertices, singular is vertex) and edges (or arcs)
  - Each **node/vertex** contains an element
  - **Edge/arc**: A pair of vertices representing a connection between two nodes in a graph
  - Each edge connects two nodes together (or possibly the same node to itself) and may contain an edge attribute

Applications of Graphs

- Cities Map
- Facebook
- Computer networks
- Task of projects

Graphs

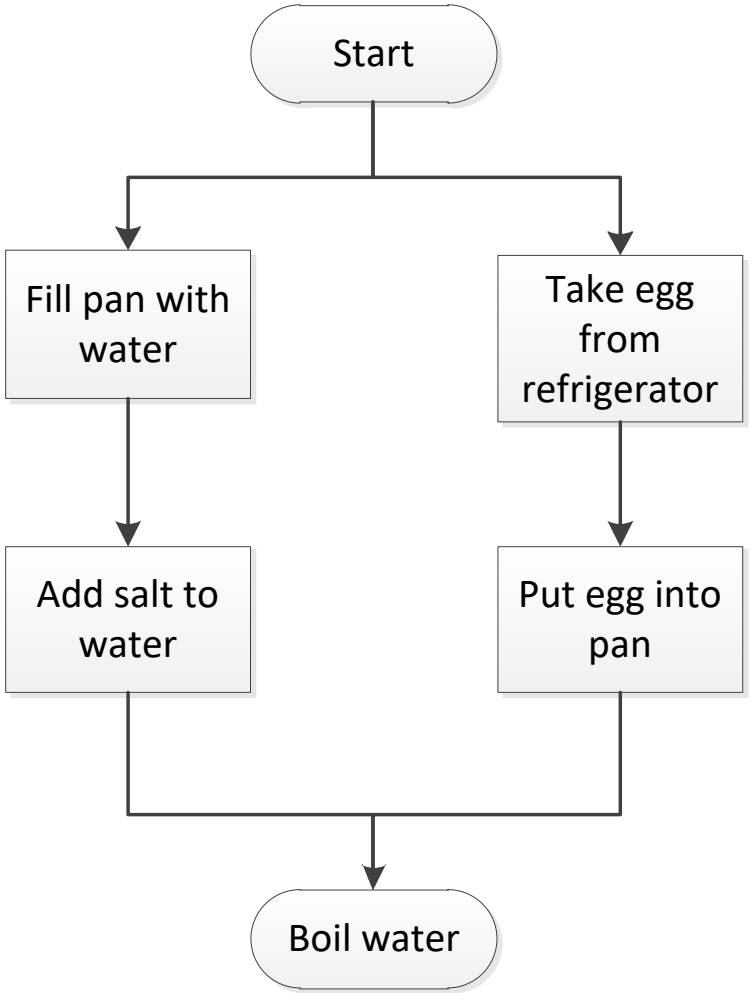
- The Graph Data Structure
  - **Set** V of vertices
  - **Collection** E of edges (pairs of vertices in V)
- Drawing of a Graph
  - Vertex →Circle/Oval
  - Edge → Line connecting the vertex pair
- V(G) is a finite, nonempty set of vertices
- E(G) is a set of edges (written as pairs of vertices)

Directed And Undirected Graphs

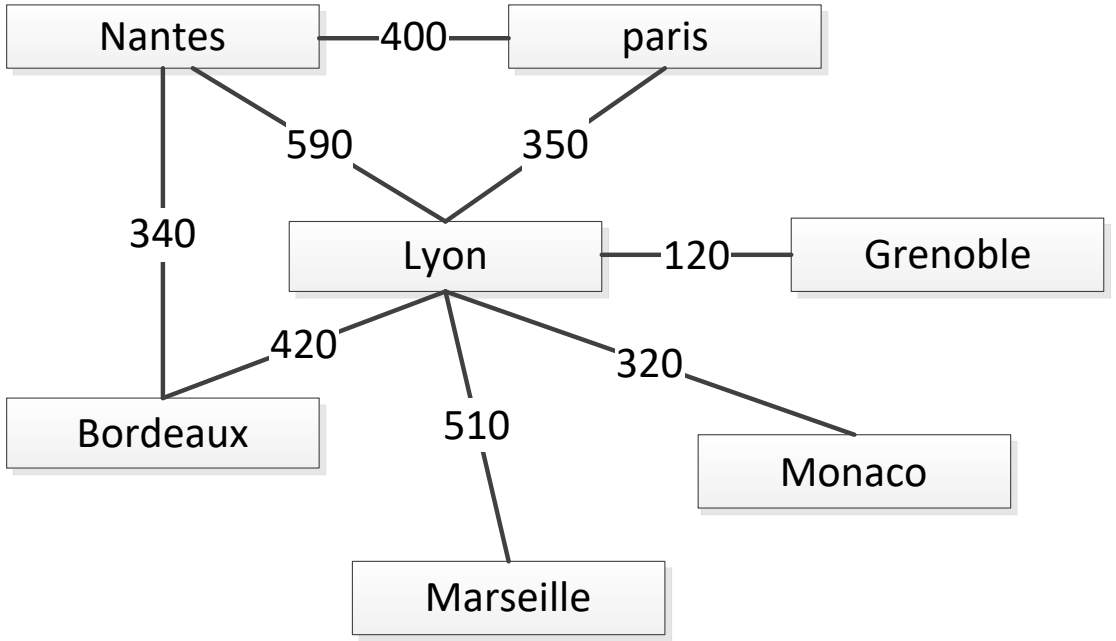
- There are two kinds of graphs: directed graphs (sometimes called digraphs) and undirected graphs
  - A **directed graph/digraph** is one in which the edges have a direction
  - An **undirected graph** is one in which the edges do not have a direction

Directed And Undirected Graphs

A directed graph



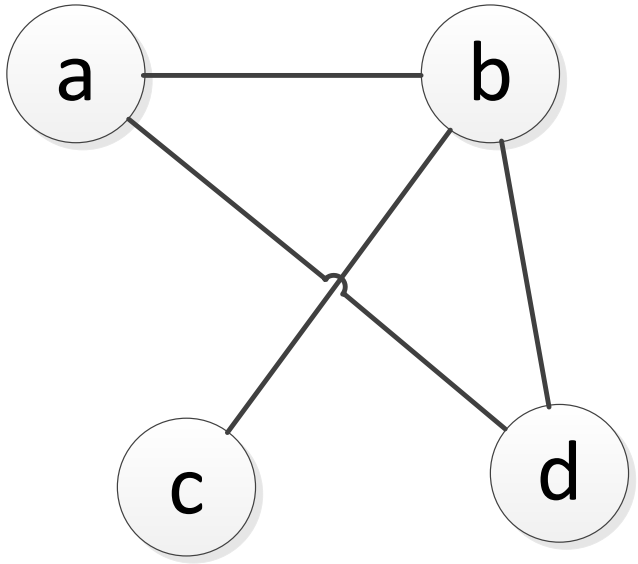
An Undirected graph



Undirected Graph Example

- Graph G=(V,E):

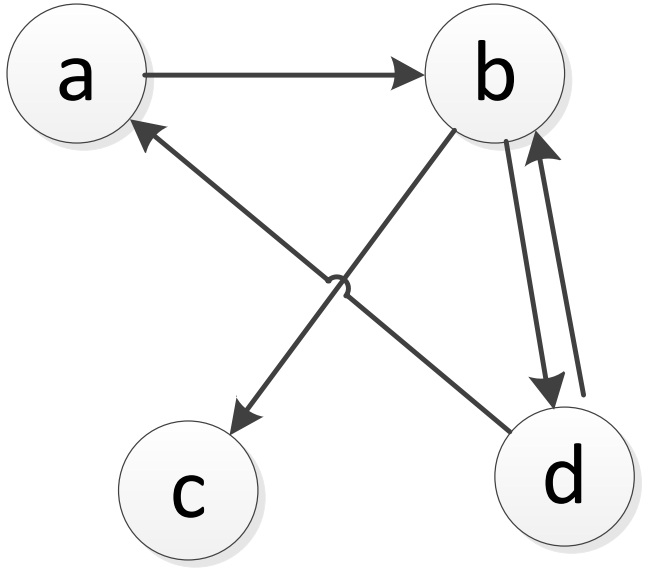
  V={a,b,c,d}, E={(a,b),(b,c),(b,d),(a,d)}



Directed Graph

- Graph G=(V,E):

  V={a,b,c,d}, E={(a,b),(b,c),(b,d),(d,b),(d,a)}

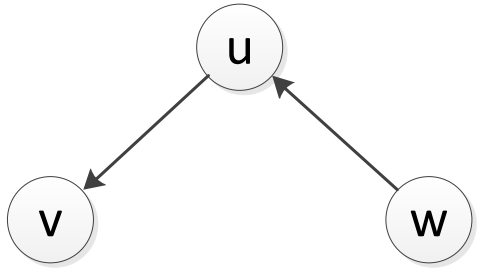


Size and Degree

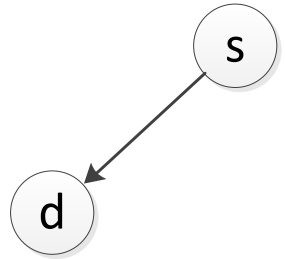
- The **size** of a graph is the number of *edges* in it
- The **empty graph** has n nodes and no edges
- If two nodes are connected by an edge, they are **neighbors** (and the nodes are adjacent to each other)
- The **degree** of a node is the number of edges it has, with loops counted twice

For directed graphs

- In a directed graph vertex v is adjacent from u, if there is an edge leaving u and coming to v. and u is adjacent to v
  - v is adjacent from u.
  - u is adjacent to v
  - u is adjacent from w
  - w is adjacent to u



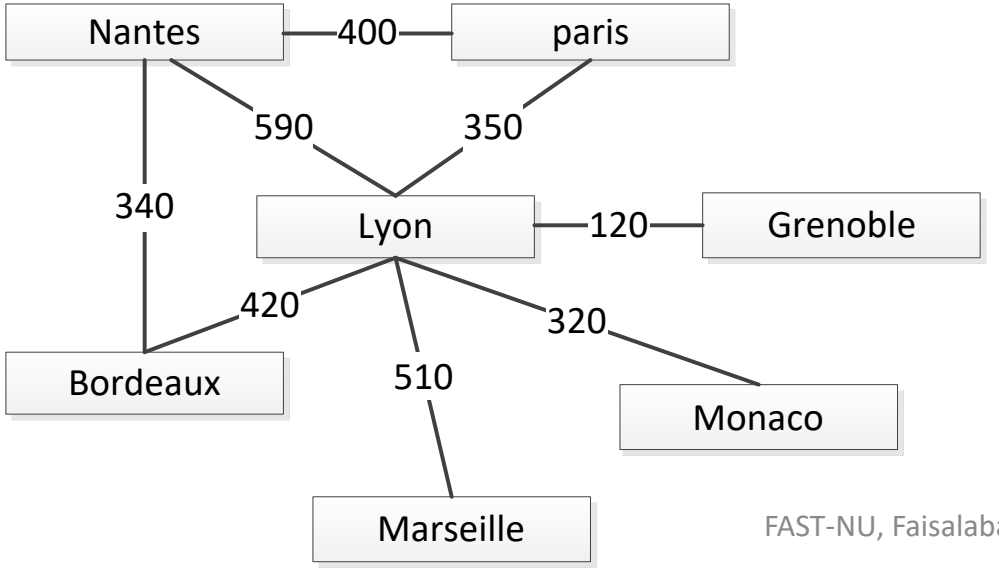
- If a directed edge goes from node S to node D, we call S the source and D the destination of the edge
  - The edge is an **out-edge** of **s** and an **in-edge** of **d**
  - **s** is a **predecessor** of **d**, and **d** is a **successor** of **s**



- The **in-degree** of a node is the number of **in-edges** it has
- The **out-degree** of a node is the number of **out-edges** it has

Path and Cycle

- In graph theory, a path in a graph is a sequence of edges which connect a sequence of vertices. Path has a first vertex, called its *start vertex*, and a last vertex, called its *end vertex*. Both of them are called *terminal vertices* of the path.
- A **simple cycle** is a **cycle** that has no repeated vertices other than the start and end vertices being the same



- Example: (Paris, Nantes, Bordeaux, Lyon)is a path
- Example: (Paris, Nantes, Lyon, Paris)is a cycle
- A cyclic graph contains at least one cycle
- An acyclic graph does not contain any cycles

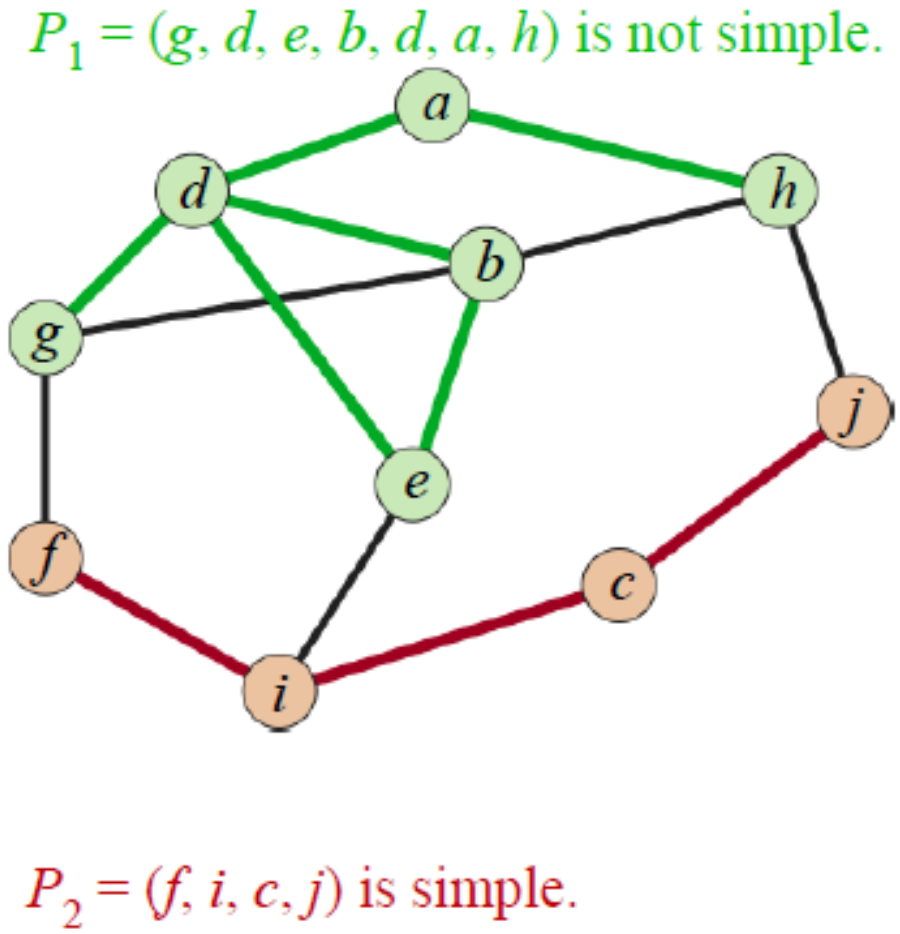
Paths and Cycles

- ***A path*** is a sequence of vertices

$P = (v_0, v_1, \ldots, v_k)$ such that,

for $1 \le i \le k$, edge $(v_{i-1}, v_i) \in E$.

Path P is ***simple*** if no vertex appears more than once in P.

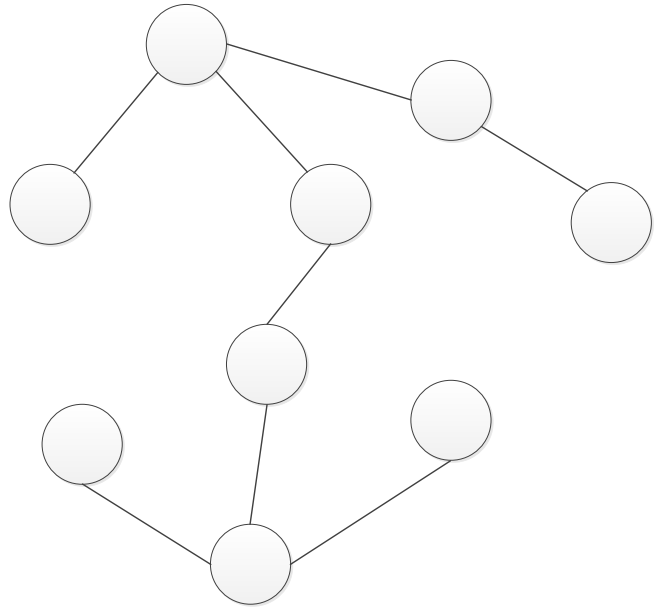


Graph Terminologies

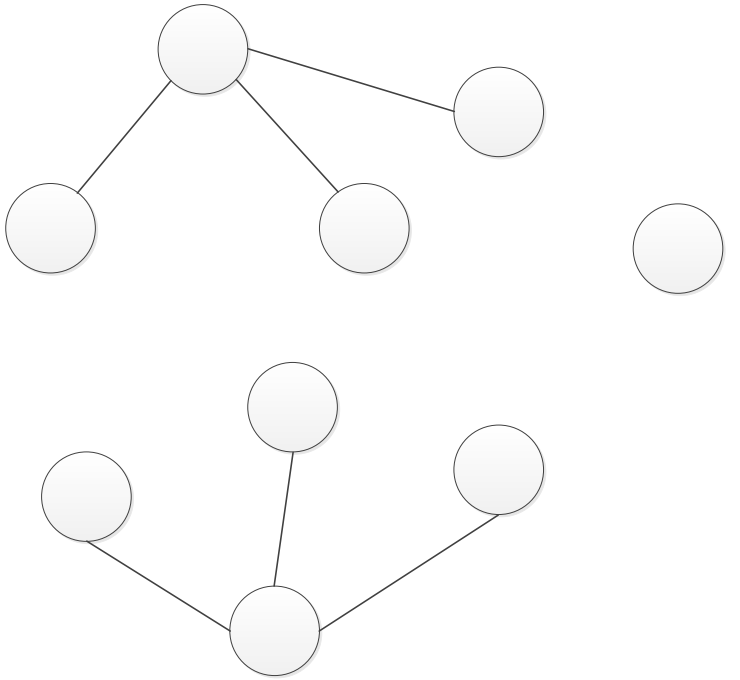
- **(simple) Path**
  - Path P is simple if no vertex appears more than once in P.
  - *Length*= # edges in the path
- A ***loop*** is an edge from a vertex onto itself.
- **(simple) Cycle**
  - Same as path except that first and last vertices are identical
- **Directed paths and cycles**
  - Edge directions matter

Trees and Forests

- A ***tree*** is a graph that is connected and contains no cycles.

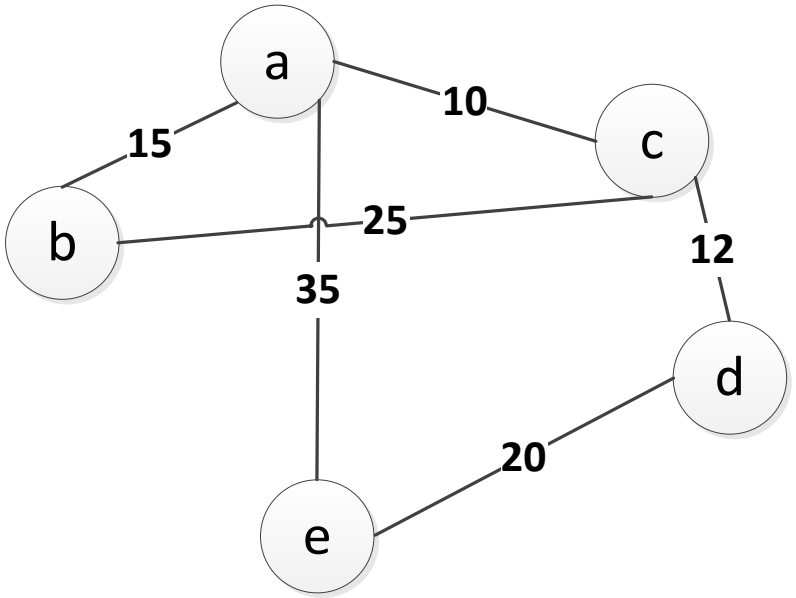


- A ***forest*** is a graph that contains no cycles.
- The connected components of a forest are trees.



Weighted Graphs

- Graph G = (V,E) such that there are weights/costs associated with each edge
  - w((a,b)): cost of edge (a,b)
  - Representation: matrix entries → costs



$$\begin{bmatrix} 0 & 15 & 10 & \propto & 35 \\ 15 & 0 & 25 & \propto & \propto \\ 10 & 25 & 0 & 12 & \propto \\ \propto & \propto & 12 & 0 & 20 \\ 35 & \propto & \propto & 20 & 0 \end{bmatrix}$$

Implementation of Graph

Graph ADT Specification

- Operations: These are the most basic functions of graph
  - MakeEmpty()
    - *Initializes the graph to an empty state.*
  - **AddVertex** (VertexType vertex)
    - *Add vertex to graph*
  - **AddEdge**(VertexType fromVertex, VertexType toVertex, int weight)
    - *Adds an edge with the specified weight from fromVertex to toVertex.*
  - **Int Weight**(VertexType fromVertex, VertexType toVertex)
    - *Determines the weight of the edge from fromVertex to toVertex.*
  - **GetToVertices**(VertexType vertex, QueType& vertexQ)
    - Returns a queue of the vertices that are adjacent from vertex.

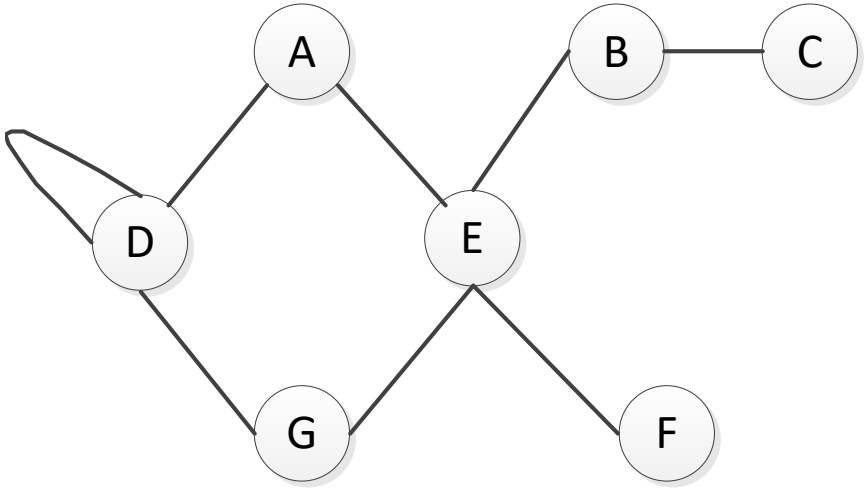
Graph Representation

- Adjacency-matrix Representation
- Adjacency Lists Representation

Representing Graphs: Adjacency Matrix

- Adjacency Matrix
  - Two-dimensional matrix of size n x n where n is the number of vertices in the graph
  - a[i, j] = 0 if there is no edge between vertices i and j
  - a[i, j] = 1 if there is an edge between vertices i and j
  - Undirected graphs have both a[i, j] and a[j, i] = 1 if there is an edge between vertices i and j
  - a[i, j] = weight for weighted graphs

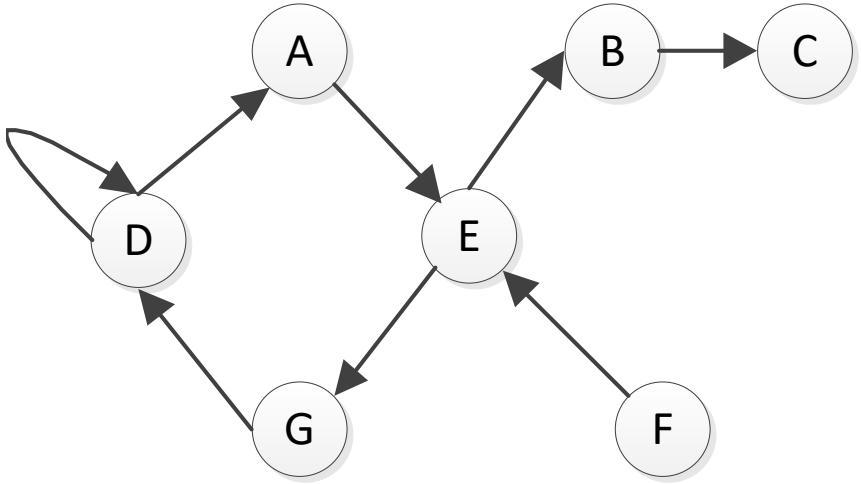
Adjacency-matrix representation I



	A	B	C	D	E	F	G
A				●	●		
B			●		●		
C		●					
D	●			●			●
E	●	●				●	●
F					●		
G				●	●		

- One simple way of representing a graph is the adjacency matrix
- A 2-D array has a mark at [i][j] if there is an edge from node i to node j
- The adjacency matrix is symmetric about the main diagonal
- This representation is only suitable for small graphs!(Why?)

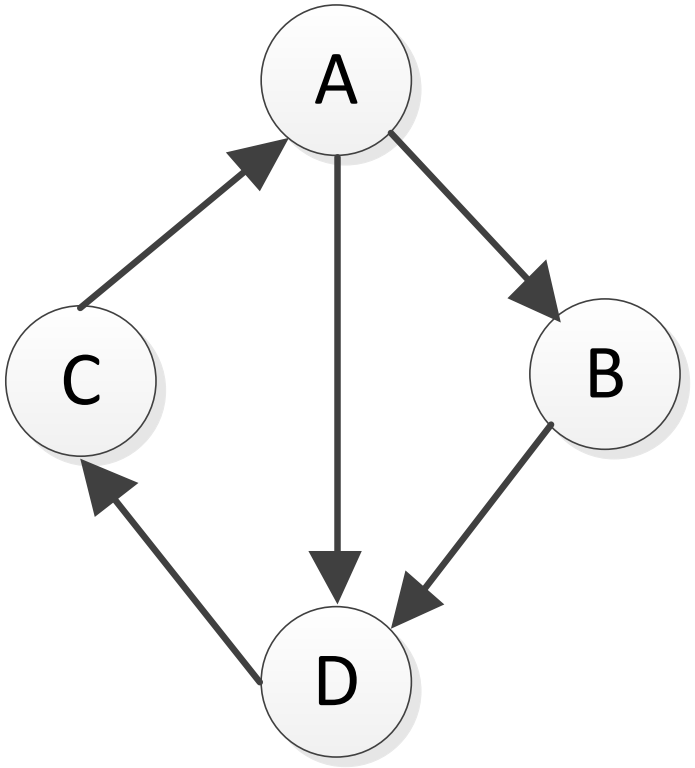
Adjacency-matrix representation II



	A	B	C	D	E	F	G
A					•		
B			•				
C							
D	•			•			
E		•					•
F					•		
G				•			

- An adjacency matrix can equally well be used for digraphs (directed graphs)
- A 2-D array has a mark at [i][j] if there is an edge from node i to node j
- Again, this is only suitable for *small* graphs!

Adjacency Matrix –Un weighted graph Example



	A	B	C	D
A	0	1	0	1
B	0	0	0	1
C	1	0	0	0
D	0	0	1	0